

DubzChain Launch Recipes

Phase 6.2 — Clean launch recipes for bootstrap, peer join, public seed node, restart/recovery, and snapshot-assisted sync.

How to use this guide

- Replace **YOUR_PUBLIC_IP** and **YOUR_BOOTSTRAP_IP** with real values before launching a public node.
- Keep **--rpc-host 127.0.0.1** unless you intentionally want RPC exposed.
- After every launch, verify **/health**, **/status**, **/peers**, **/sync**, and **/index**.

Recipe A — Local single mining node

Use this for local testing and explorer work.

```
node dist/index.js 3001 --host 127.0.0.1 --rpc-host 127.0.0.1 --automine --mine-empty
```

Expected

- Explorer should load at <http://127.0.0.1:4001/index>
- **/health** should return ok
- A local miner wallet file should exist
- Blocks should begin mining automatically

Pass condition

- RPC responds
- Explorer loads
- Height increases over time

Recipe B — Public bootstrap node

Use this for the main public node that other peers join first.

```
node dist/index.js 3001 --host 0.0.0.0 --rpc-host 127.0.0.1 --advertise ws://YOUR_PUBLIC_IP:3001 --au
```

Expected

- Public P2P port is reachable
- Explorer is available locally on port 4001
- Node advertises the public websocket URL correctly
- Peers can use it as their first join point

Pass condition

- **/peers** eventually shows connected nodes
- **/status** returns healthy chain state

- Remote nodes sync from it

Recipe C — Public peer node joining bootstrap

Use this for a second public node that joins from the bootstrap node.

```
node dist/index.js 3002 --host 0.0.0.0 --rpc-host 127.0.0.1 --peer ws://YOUR_BOOTSTRAP_IP:3001 --adve
```

Expected

- Node connects to the bootstrap node
- Headers and block sync begin
- Peer table begins populating
- Explorer is available locally on port 4002

Pass condition

- /sync shows progress
- Height catches up to bootstrap
- /peers shows at least one live connection

Recipe D — Public seed node

Use this for a stable relay/discovery node. Mining is optional.

```
node dist/index.js 3001 --host 0.0.0.0 --rpc-host 127.0.0.1 --advertise ws://YOUR_PUBLIC_IP:3001
```

Expected

- Node accepts public P2P connections
- Peer discovery works through this node
- Node remains stable even if not mining

Pass condition

- Other nodes can connect
- /peers stays active
- /health and /status remain healthy

Recipe E — Snapshot-assisted join

Use this when joining a node that may import a snapshot and continue normal sync.

```
node dist/index.js 3004 --host 0.0.0.0 --rpc-host 127.0.0.1 --peer ws://YOUR_BOOTSTRAP_IP:3001 --adve
```

Expected

- Snapshot metadata may be requested first
- Snapshot may import before full block catch-up finishes
- Node should continue normal block sync after snapshot load

Pass condition

- /sync progresses after snapshot stage
- Node reaches current tip
- /debug/replay-verify remains valid

Recipe F — Restart / recovery

Use the same launch command that was used originally after stopping the node cleanly.

Restart with the original command for that node role.

Expected

- Chain data should reload
- Peer table should reload
- Explorer and RPC should return
- Node should resume mining or syncing normally

Pass condition

- /health returns ok after restart
- Tip height is preserved or continues growing
- /peers reconnects successfully

Post-launch checks

- Open /health for a quick ok/not-ok check.
- Open /status for chain height, tip hash, peers, and memory summary.
- Open /peers to confirm live connections.
- Open /sync to confirm the node is catching up or fully caught up.
- Open /index to confirm the explorer is healthy.
- Run /debug/replay-verify after restarts or upgrades.

Notes

If your runtime has explicit prune or archival flags, add those exact commands to this guide before marking 6.2 final. The recipes above are clean baseline launch flows based on the commands already used in your project.